🎬 Guion de Video: Sustentación Técnica y Funcional — Proyecto PEA-i (Khemia)

- **Asignatura:** Estructura de Datos (Grupo 01)
- **Docente:** Ing. Adith Bismarck Pérez Orozco
- **Institución:** Universidad Popular del Cesar (UPC) — Facultad de Ingenierías y Tecnológicas
- **Programa:** Ingeniería de Sistemas
- **Grupo de Trabajo:** `WP - DN - EO` (Grupo 5)
- **Duración estimada:** 10 a 12 minutos
- **Formato recomendado:** Grabación de pantalla a 1080p (OBS Studio) con cámara de los integrantes visible durante toda la grabación (requisito formal de rúbrica).

📋 Preparación previa a la grabación (Checklist Técnico)

1. **Terminal 1 (Consola Autónoma C++):**
   - Tener abierta una ventana de PowerShell en la raíz con el comando listo para ejecutar:

     ```
     .\core_cpp\build\Release\pea_cli.exe summary
     ```

2. **Terminal 2 (Servidor FastAPI & Base de Datos):**
   - SQL Server activo (`Database=peai`) y servidor uvicorn corriendo:

     ```
     uvicorn backend.app.main:app --port 8000
     ```

   - En la consola de inicio debe verse que `abpoxx_pybind.pyd` montó exitosamente en la memoria RAM de C++ los grupos insignias de la UPC: **AITICE (`COL0043834`, Cat. B)** y **GISICO (`COL0002099`, Cat. C)**.
3. **IDE (VS Code) con pestañas clave preparadas para zoom:**
   - `core_cpp/include/entities/group.h` (Estructura de nodos de la Multilista).
   - `core_cpp/include/entities/membership.h` (Nodos ortogonales de relación grupo-investigador).
   - `core_cpp/include/services/validation_queue.h` (Cola FIFO en memoria dinámica).
   - `core_cpp/include/services/undo_stack.h` (Pila LIFO de control de cambios).
   - `backend/app/evaluator/tier_rules.py` (Motor de auditoría de MinCiencias 2024).
4. **Navegador Web:**
   - Frontend React abierto en `http://localhost:5173` con sesión activa en el Dashboard Analítico, vistas de grupos, cola de validación e importador.
5. **Diagramas de Arquitectura listos en visor:**
   - Galería `docs/diagrams/`: `03_multilista_memoria_ram`, `05_hipercubo_informacion`, `09_secuencia_validacion_fifo`, `10_secuencia_deshacer_lifo` y `11_arquitectura_despliegue`.

⏱️ Escena 1: Introducción, Identidad Institucional e Interoperabilidad (00:00 - 01:45)

- **Presentador:** Integrante 1 (`WP`)
- **En pantalla:** Portada institucional con logo de la UPC, cambio rápido a VS Code mostrando el binario compilado `abpoxx_pybind.pyd` y luego el diagrama `11_arquitectura_despliegue`.
- **Cámara:** Los 3 estudiantes visibles en cuadro.

🎙️ Locución (Integrante 1):

> "Un cordial saludo, Ingeniero Adith Bismarck Pérez Orozco. Somos el Grupo 5 de la asignatura Estructura de Datos de la Universidad Popular del Cesar, conformado por [Nombre WP], [Nombre DN] y [Nombre EO]. A continuación sustentamos nuestro proyecto **PEA-i (Khemia)**: la plataforma integral de analítica científica para la gestión de grupos de investigación, investigadores y categorización institucional.
>
> *(Mostrar diagrama 11_arquitectura_despliegue y el archivo abpoxx_pybind.pyd en VS Code)* Para resolver el Taller 2 no diseñamos un sistema web convencional. Adoptamos el desafío más exigente de la rúbrica: el **Punto 14.d de Interoperabilidad e integración entre lenguajes**. Desarrollamos un **núcleo de alto rendimiento en C++** donde todas las estructuras de datos (Multilistas, Pilas, Colas e Hipercubo) operan directamente en memoria RAM mediante punteros manuales.
>
> Este núcleo fue compilado como una librería nativa con **Pybind11**, exponiéndose directamente hacia un backend en **FastAPI (Python)** y consumiéndose por una interfaz web reactiva en **React y TypeScript**. Además, garantizamos un sistema de **escritura dual write-through**, donde cada cambio en la RAM se replica atómicamente tanto en **SQL Server** como en nuestro archivo serializado `pea_data.json`."

⏱️ Escena 2: Consola C++ Autónoma y Multilista Ortogonal en RAM (01:45 - 04:30)

- **Presentador:** Integrante 1 (`WP`)
- **En pantalla:**
  1. Terminal de consola ejecutando `pea_cli.exe summary` (Punto 12.a).
  2. VS Code: `core_cpp/include/entities/group.h` y `membership.h` (Punto 8).
  3. Navegador: Vista de Grupos y Miembros en React.

🎙️ Locución (Integrante 1):

> "Cumpliendo estrictamente con el **Punto 12.a del taller**, donde se exige que la solución en C presente la información en tipo resumen de datos y tablas sin depender de interfaces gráficas, pasemos primero a la terminal.
>
> *(En la terminal PowerShell, ejecutar: `.\core_cpp\build\Release\pea_cli.exe summary`)* Como se puede observar en la consola, nuestro ejecutable de C++ es 100% autónomo. Aquí genera la tabla formateada con las métricas institucionales de la UPC: grupos activos, investigadores, membresías vinculadas y el total de productos en RAM sin requerir el servidor web.

> *(Cambiar a VS Code en `core_cpp/include/entities/group.h` y `membership.h`)* Ahora veamos el corazón algorítmico del **Punto 8: el diseño de la Multilista en Memoria RAM**. En `group.h` y `membership.h` podemos evidenciar los punteros directos:
>
> - `GroupNode* next`: mantiene la lista simplemente enlazada de grupos de investigación.
> - `MembershipNode* head_member`: puntero ortogonal que ancla la sublista de investigadores adscritos al grupo.
> - `GroupProductNode* head_product`: puntero ortogonal cruzado hacia el catálogo de productos. Cada nodo se reserva dinámicamente con operadores de memoria en el Heap sin utilizar contenedores automáticos prefabricados.
>
> *(Cambiar al navegador web en la pestaña de Grupos e Investigadores)* En la interfaz gráfica vemos reflejada esta estructura en caliente con los grupos reales de la UPC que tenemos precargados: **AITICE (`COL0043834`)** y **GISICO (`COL0002099`)**. Al consultar o editar un integrante, el motor en C++ aplica las reglas de negocio: valida que no existan traslapes de fechas inválidos, controla el rol del investigador y permite la **desactivación lógica (`activo = 0`)** requerida en el Punto 9 sin romper los punteros de la multilista."

⏱️ Escena 3: Taxonomía MinCiencias 2024, Ingesta Multifuente y Cola FIFO (04:30 - 07:15)

- **Presentador:** Integrante 2 (`DN`)
- **En pantalla:**
  1. VS Code: `core_cpp/include/services/validation_queue.h`.
  2. Navegador: Módulo de Ingesta (URL, CSV, PDF) y Bandeja de la Cola de Validación.
  3. Diagrama `09_secuencia_validacion_fifo`.

🎙️ Locución (Integrante 2):

> "Muchas gracias. Abordando el **Punto 5 (Modelo de Medición MinCiencias 2024)** y el **Punto 7 (Ingesta Multifuente)**, implementamos la taxonomía oficial de la Convocatoria 957:
>
> A diferencia de aproximaciones que simplifican el problema en 15 tipos viejos de GrupLAC, nuestro sistema incorpora las **5 Macrofamilias oficiales completas**:
>
> 1. **GNC:** Generación de Nuevo Conocimiento.
> 2. **DTI:** Desarrollo Tecnológico e Innovación.
> 3. **ASC:** Apropiación Social del Conocimiento.
> 4. **DPC:** Divulgación Pública de la Ciencia.
> 5. **FRH:** Formación de Recurso Humano. Clasificadas sobre **70 subtipos canónicos del Anexo 1** y distinguiendo el nivel doctoral en FRH-A.
>
> *(Mostrar en el navegador el módulo de Ingesta)* Para el **Punto 7**, nuestra plataforma soporta las 3 fuentes requeridas:
>
> - Scraping por URL directa desde la plataforma Scienti (GrupLAC y CvLAC) con detección de aval institucional mediante `chulo_1.jpg`.
> - Importación de datasets CSV de Datos Abiertos Colombia.
> - Extracción de reportes oficiales en formato PDF.

> *(Cambiar a VS Code en `core_cpp/include/services/validation_queue.h`)* Cuando un producto ingresa al sistema, no pasa de inmediato a las métricas del grupo. Aquí entra en juego el **TDA Cola (FIFO — First-In, First-Out)** programado en C++. En `validation_queue.h` observamos la estructura con punteros `QueueNode* front` y `QueueNode* rear`, garantizando inserción (`enqueue`) y extracción (`dequeue`) en tiempo constante $\mathcal{O}(1)$.
>
> *(Mostrar en la interfaz web la Bandeja de Validación y procesar un elemento)* En la pantalla de validación vemos la lista ordenada cronológicamente: el primer producto radicado por un docente es exactamente el primero que el evaluador atiende. Al hacer clic en 'Aprobar', el producto se desencola y sus punteros se integran a la Multilista activa."

⏱ Escena 4: Pila LIFO de Deshacer (*Undo*) y Control de Cambios (07:15 - 09:15)

- **Presentador:** Integrante 2 (`DN`)
- **En pantalla:**
  1. VS Code: `core_cpp/include/services/undo_stack.h`.
  2. Navegador: Realizar una modificación/eliminación en vivo y presionar el botón **Deshacer (Undo)**.
  3. Diagrama `10_secuencia_deshacer_lifo`.

🎙 **Locución (Integrante 2):**

> "Para satisfacer el requerimiento de reversión y control de cambios en memoria, implementamos el **TDA Pila (LIFO — Last-In, First-Out)** en C++ (Diagrama 10).
>
> *(Mostrar en VS Code el archivo `undo_stack.h`)* En `undo_stack.h` definimos la estructura `UndoOperation`. Cada operación mutante sobre la multilista (creación, edición, vinculación o borrado lógico) apila (`undo_push`) una instantánea con el estado previo y la acción inversa correspondiente en $\mathcal{O}(1)$.
>
> *(Demostración en vivo en el navegador)* Hagamos la prueba en tiempo real: Vamos a modificar la categoría de este producto en el grupo AITICE de 'B' a 'A1'. Observen cómo el cambio se refleja al instante en la tabla y en el contador de la base de datos.
>
> Si el usuario cometió una equivocación, simplemente pulsa el botón **'Deshacer'**. En este momento, la petición viaja por la API hacia C++, invoca la función `undo_perform()`, desapila (`undo_pop`) la última operación LIFO, revierte los punteros en la RAM al estado anterior y sincroniza la persistencia. La categoría retorna automáticamente a 'B', demostrando integridad transaccional perfecta."

⏱ Escena 5: Hipercubo 4D, Ventanas de Observación y Auditoría MinCiencias (09:15 - 11:15)

- **Presentador:** Integrante 3 (`EO`)
- **En pantalla:**
  1. Diagrama `05_hipercubo_informacion`.
  2. Navegador: Dashboard Analítico de Khemia, interactuando con los filtros de ventana (2 y 5 años) y mostrando el Semáforo de Tiers.

3. VS Code: `backend/app/evaluator/tier_rules.py`.

Locución (Integrante 3):

> "Llegando a la analítica avanzada, el taller solicita modelar la información bajo la filosofía de un **Hipercubo Multidimensional (Punto 12.b y 12.c)**.
>
> *(Mostrar diagrama 05_hipercubo_informacion y luego el Dashboard web interactivo)* Nuestro Hipercubo indexa las métricas en 4 dimensiones ortogonales:
>
> 1. **Dimensión Entidad:** Universidad Popular del Cesar $\rightarrow$ Grupos $\rightarrow$ Investigadores.
> 2. **Dimensión Tipología:** 5 Familias y 70 Subtipos oficiales.
> 3. **Dimensión Validación:** Pendiente, Válido o Rechazado.
> 4. **Dimensión Temporal:** Ventanas de observación.
>
> *(Interactuar con el selector de ventana de tiempo en el Dashboard)* En atención al **Punto 10 del taller**, el usuario puede aplicar filtros deslizantes:
>
> - Al seleccionar **'Últimos 2 años'**, el dashboard aísla la producción 2024-2026 para convocatorias de respuesta rápida.
> - Al alternar a **'Últimos 5 años'**, proyecta la ventana quinquenal estándar (2021-2026). Pero además, nuestro motor aplica las **ventanas diferenciadas del modelo 2024**: reconoce que las patentes y variedades vegetales tienen vigencia de 10 años, y los artículos y libros de 7 años.
>
> *(Mostrar el panel del Semáforo MinCiencias del grupo AITICE)* En esta vista presentamos el **Motor de Auditoría de Tiers MinCiencias**: Calculamos matemáticamente el **Índice de Cohesión (I_{coh})** analizando la coautoría interna entre docentes de la UPC en obras GNC/DTI. El sistema verifica si el grupo cumple la antigüedad, integrantes mínimos, proyectos I+D activos y umbrales de calidad para clasificar en **Reconocido, C, B, A o A1**, presentándole al coordinador institucional una lista de chequeo de lo que le falta al grupo para ascender."

Escena 6: Conclusiones, Entregables y Cierre (11:15 - 12:00)

- **Presentador:** Integrante 3 (`EO`)
- **En pantalla:** Vista del documento Word `Taller2_WP-DN-EO.docx`, el repositorio en GitHub y los 3 estudiantes en cámara completa despidiéndose.

Locución (Integrante 3):

> "En conclusión, con **PEA-i (Khemia)** hemos demostrado el cumplimiento integral de todos y cada uno de los puntos exigidos en el Taller 2 de Estructura de Datos:
>
> - Modelado en **Multilistas dinámicas**, **Pila LIFO de Undo**, **Cola FIFO de validación** e **Hipercubo analítico** con memoria dinámica en **C++**.
> - Consola descriptiva autónoma en **C++** y Dashboard gráfico interactivo en **React**.
> - **Interoperabilidad nativa** mediante **Pybind11 (Punto 14.d)**.
> - Persistencia dual en **SQL Server** y archivo serializado **JSON**.
> - Rigor absoluto con la **Convocatoria 957 de MinCiencias** y datos reales de la Universidad Popular del Cesar.

> Toda la especificación formal, diagramas UML y casos de prueba se encuentran compilados en el documento técnico Word `Taller2_WP-DN-EO.docx` adjunto al correo de entrega.
>
> Muchas gracias por su atención, Ingeniero Adith."

💡 Criterios Clave para una Grabación de 5.0 (Recomendaciones del Equipo)

1. **Visibilidad en cámara (Obligatorio por rúbrica):** Asegurarse de que los 3 integrantes tengan la cámara encendida durante toda la grabación en una esquina (OBS con cámara compuesta o llamada en Meet/Teams grabada).
2. **Terminal C++ a primera vista:** Dedicarle los 20 segundos de la Escena 2 a correr `pea_cli.exe summary`. Eso neutraliza de inmediato cualquier duda del profesor sobre si el proyecto depende exclusivamente de la web.
3. **Punteros explícitos:** Cuando abran `group.h` y `membership.h`, subrayen con el cursor los asteriscos de los punteros (`*head_member`, `*head_product`). El profesor de Estructura de Datos evalúa la gestión de memoria dinámica en el Heap.
4. **Nombres de grupos reales:** Usar siempre los nombres y códigos reales de la UPC (**GISICO** `COL0002099` y **AITICE** `COL0043834`), lo que demuestra autenticidad institucional.